

Zero-Shot Compositional Tool Planning for Adaptive Online Modulation Recognition with an IQ-Free LLM Controller

Min Ji, Lu Sun, Haoran Zha, Yun Lin
College of Information and Communication Engineering
Harbin Engineering University, Harbin, China

Abstract—Adaptive online automatic modulation recognition (AMR) requires both frame classification and decisions about when changing RF conditions warrant diagnosis, labeling, or model update. We formulate the second problem as IQ-free zero-shot tool planning. A large language model (LLM) controller receives only a structured RF State Card and a textual API for deterministic tools; it selects tools for accuracy checking, drift detection, confusion analysis, open-set rejection, adaptation, and label-budget allocation. The controller does not observe IQ samples or modulation labels, and all measurements, thresholds, and numerical actions are executed by the CNN recognizer and RF tools. On 78 adaptive AMR intents, five 7B–30B models achieve tool-set F1 up to 0.921 on held-out compound scenarios, compared with 0.567 for the best trained router and 0.649 for a compound-trained router. Blind relabeling, dataset transfer, and cross-model scenario generation indicate that the improvement is not tied to a single annotation source. Closed-loop trials expose a boundary: LLMs are useful for diagnostic orchestration, but thresholded corrective actions should remain deterministic. The result is an auditable control interface for adaptive online AMR.

Index Terms—automatic modulation recognition, LLM controller, zero-shot tool planning, adaptive signal processing, cognitive radio.

I. INTRODUCTION

Automatic modulation recognition (AMR) enables cognitive and non-cooperative receivers to infer modulation format without transmitter-side information. Deep classifiers have substantially improved closed-set AMR: convolutional and recurrent models reach 92–95% accuracy on RML2016.10A [1]–[3], and physical-layer signal-recognition studies have examined image-based representations, real-world RF data, and dedicated AMR architectures [4]–[6]. Field deployment, however, introduces a control layer beyond classification. A receiver must detect unreliable recognition, diagnose SNR loss, drift, open-set traffic, calibration error, or label imbalance, and select a corrective operation under budget.

Existing mechanisms cover isolated parts of this loop, not the controller. Active learning improves label efficiency under distribution shift [7], yet its acquisition rule is usually fixed before deployment. Open-set rejection can flag unknown traffic, but it does not decide whether to probe confusion, allocate labels, or adapt the recognizer. Recent RF-LLM and RF-VLM work often uses a foundation model to classify signal tokens or spectrograms [8]–[10]. Online AMR instead needs supervisory

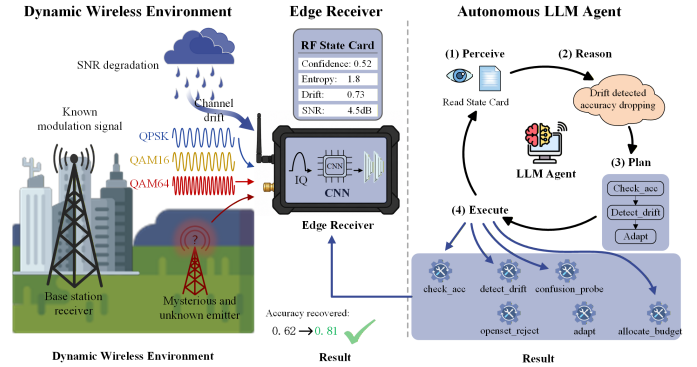


Fig. 1. Adaptive online AMR. IQ streams are classified by a CNN. The LLM controller reads only the RF State Card, uses adaptive-state summaries to plan a tool chain, and delegates numeric actions to verified RF tools.

control: under compound faults and operator constraints, the system must compose diagnostic and corrective tools that were not observed as one training label.

We study the LLM as a tool-planning controller, not as an AMR classifier. A CNN remains the sole recognizer, and deterministic tools encapsulate signal processing, thresholds, and numeric decisions. The LLM reads only an IQ-free RF State/Candidate Card and a textual tool API, then selects tools to invoke (Fig. 1). This setting tests whether language models can compose symbolic actions over verified RF modules [11]–[14], while measurements and thresholded actions remain deterministic. Tool-augmented LLM systems have appeared in broader wireless settings [15]–[17]; adaptive online AMR asks whether such controllers can assemble unseen tool chains rather than route intents to memorized labels [18].

Our evaluation separates three questions. Can an IQ-free controller infer the right RF tools from natural-language adaptive intents? Does the improvement persist under blind relabeling, cross-dataset execution, and scenarios authored by another model? When a closed-loop action depends on numeric tool returns, should the LLM decide the action or should deterministic rules take over?

We make three contributions.

- We define an IQ-free LLM controller over six deterministic RF tools, without waveform access or a classification role.
- On 78 scenarios, five 7B–30B models reach novel F1 0.856–0.921, exceeding router baselines (0.567/0.649).

- Relabeling ($\kappa=0.894$), dataset transfer, and cross-model scenarios test label and scenario dependence; numeric actions remain deterministic.

II. SYSTEM MODEL

A streaming AMR receiver processes non-stationary IQ frames affected by SNR ramps, fading, class-prior shift, and sporadic *unknown* modulations. These effects may reduce closed-set accuracy, induce silent open-set errors, or appear jointly. The control problem is to select, under a limited labeling budget, which diagnostic or corrective RF operation should be executed. Classification remains inside the CNN recognizer, aided by an energy/prototype OOD score; the LLM receives neither waveforms nor class-label observations.

At episode t , the IQ-free card is a monitoring vector rather than a signal representation:

$$c_t = (\hat{\mu}_{\text{SNR}}, \hat{\sigma}_{\text{SNR}}, d_t, \text{ECE}_t, \mathbf{n}_t, B_t, N_t^u, \bar{h}_t, \bar{m}_t, g_t, \mathcal{D}_t),$$

$$\text{ECE}_t = \sum_{b=1}^M \frac{|I_b|}{N_t} |\text{acc}(I_b) - \text{conf}(I_b)|. \quad (1)$$

Here d_t is a drift proxy, $\mathbf{n}_t$ is the labeled class-count vector, B_t is the remaining budget, N_t^u is the pool size, $\mathcal{D}_t$ denotes deficit classes, and $N_t = \sum_b |I_b|$; $\text{acc}(\cdot)$ and $\text{conf}(\cdot)$ are calibration-bin accuracy and mean confidence.

The controller interface is an IQ-free textual summary (Table I). The *State Card* reports step, SNR statistics, drift level, calibration ECE, per-class labeled counts, and remaining budget. The *Candidate Card* describes the unlabeled pool through size, entropy, margin, feature-coverage gap, and under-represented classes. Energy-OOD and prototype-distance statistics remain inside REJECTOPENSET. In simulation, *drift-level* is regime-derived; in deployment, DETECTDRIFT provides a reference-batch accuracy-drop proxy. Secs. IV-A and IV-B use only the operator intent and tool API; Sec. IV-C adds the cards for closed-loop execution.

Six deterministic RF tools define the action space and return compact textual observations (Table II). The LLM therefore observes tool outcomes, not IQ samples or floating-point feature vectors.

III. PROPOSED METHOD

We formulate the controller as tool-conditioned planning over a verified RF API [13], [14]. Let $\mathcal{T} = \{t_j\}_{j=1}^6$ denote the deterministic tools, q the operator intent, c_t the State Card, and H_t the history of tool returns. The LLM induces a bounded sequential policy

$$\pi_\theta(q, c_t, H_t; \mathcal{T}) = (t_1, \dots, t_m), \quad t_i \in \mathcal{T}, \quad m \leq K, \quad (2)$$

$$P = \{t_i\}_{i=1}^m.$$

The set P may be returned directly or assembled through ReAct-style calls [12]. Each t_j encapsulates signal-processing inputs, thresholds, and numerical decisions; the LLM selects and orders calls.

We compare this planner with a fixed majority-plan baseline and six router baselines. Four *learned tool routers* are

TABLE I
RF STATE/CANDIDATE CARD FIELDS (IQ-FREE, DETERMINISTIC).

Field	Source	Example
SNR-est, SNR-std	batch estimate	6.0 dB (std 3.1)
drift-level [†]	regime / proxy	0.42 (0=stable, 1=max)
ECE	reliability bins	0.18 (miscalibrated)
class-counts	labeled buffer	[14, 0, ...] (3 unseen)
budget-left	scheduler	8/16
pool entropy/margin	unlabeled pool	0.78 / 0.31
coverage-gap, deficit	feature space	0.22, {QAM64}

TABLE II
DETERMINISTIC RF TOOL API; TOOLS EXECUTE ALL SIGNAL PROCESSING.

Tool	Tool observation (example)
CHECKACCURACY	"accuracy 0.61; ref. 0.78"
DETECTDRIFT	"covariate drift; acc. drop 0.17"
PROBECONFUSION	"QAM64↔QAM16 pair, 38%"
REJECTOPENSET	"OOD fraction 0.40; nearest QPSK"
ADAPT	"accuracy 0.61 to 0.69"
ALLOCATEBUDGET	"budget to confused pair"

trained on base intents: TF-IDF nearest-neighbor, char- n -gram nearest-neighbor, and TF-IDF or char- n -gram multi-label logistic routers. We also include a *zero-shot* tool-description router and a *semantic-embedding* router using mean-pooled 8B-model embeddings. All baselines use the same labels and tool inventory.

Adaptive scenarios are organized by compositional depth. T1 requires one tool, T2 a short sequence, T3 introduces a condition, and T4 contains held-out compounds such as {CHECKACCURACY, DETECTDRIFT, PROBECONFUSION, ADAPT}. Base scenarios are T1/T2; novel scenarios are T3/T4, where the correct plan combines tools in ways absent from router training. The split therefore tests compositional transfer rather than interpolation over seen intent labels.

Alg. 1 gives the sequential form. A short memory of (card, action) pairs can be retrieved as text, but the planning results reported below do not rely on this memory.

For the intent benchmark, which labels required tools rather than execution traces, we score plans against an authored minimal set G by

$$F_1(P, G) = \frac{2|P \cap G|}{|P| + |G|}, \quad (3)$$

averaged separately over *base* and *novel* intents. Significance uses an 8000-sample bootstrap.

IV. EXPERIMENTS AND RESULTS

The benchmark contains 78 intents: 38 base intents and 40 novel intents. We evaluate five LLM controllers: Qwen3-8B and Qwen3-30B-A3B (MoE, 3B active) [19], Mistral-7B [20], Gemma-4-12B [21], and GPT-5.5. Local models are queried at temperature 0. Closed-loop tool execution uses our online-AMR pipeline on RML2016.10A and RML2018.01A [22]. The closed-loop tests use eight hidden RML2016.10A scenarios: four base cases (drift, novelty, confusion, healthy) and four compound variants (severe drift, drift+novelty, subtle novelty, noisy healthy).

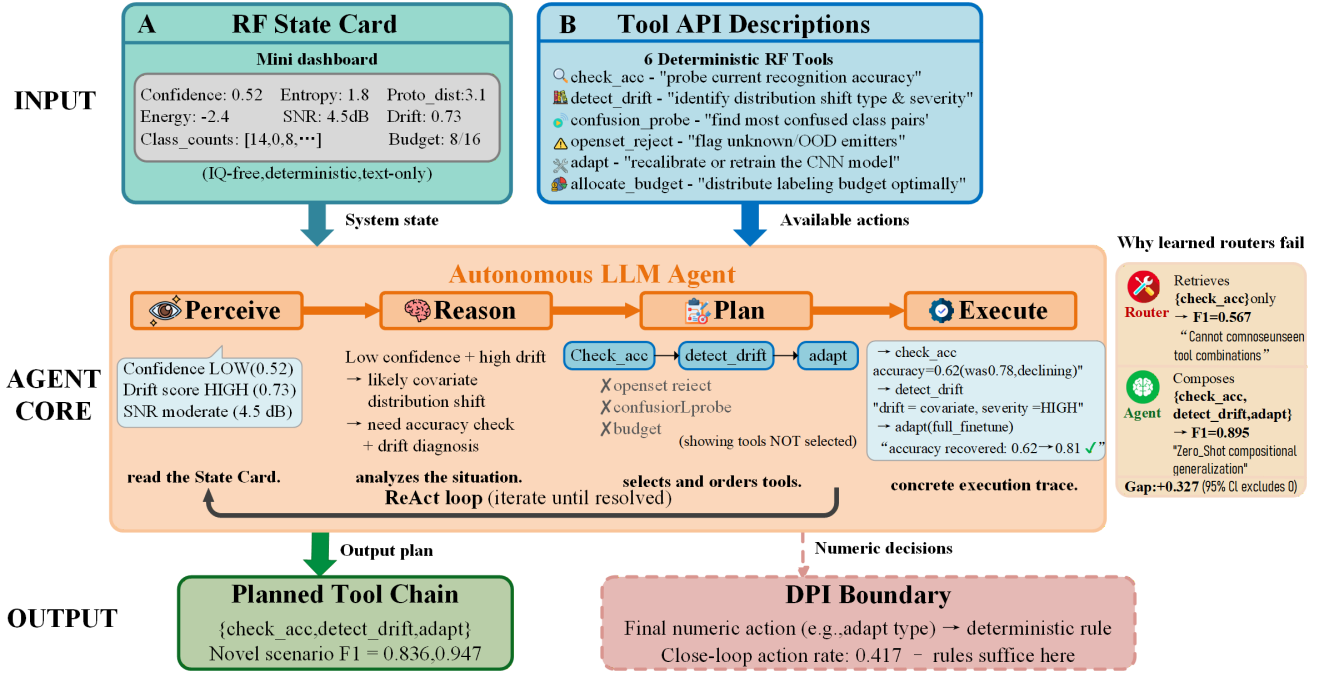


Fig. 2. Controller workflow. The LLM receives the RF State Card and tool API, plans a tool chain, and executes through ReAct. Its F1 improvement over routers (+0.289 to +0.354) comes from assembling held-out tool combinations. The rule boundary marks where deterministic decisions take over.

Algorithm 1 Controller for operator-intent tool planning

Require: intent q , tool API $\mathcal{T}$, State Card c , budget K

- 1: $H \leftarrow \emptyset$ ▷ history of tool observations
- 2: **for** $k = 1 \dots K$ **do**
- 3: $r \leftarrow \text{LLM}(\text{SYS}(\mathcal{T}), q, c, H)$ ▷ planner query
- 4: **if** $r = \text{DONE}(a)$ **then**
- 5: **return** action a
- 6: $t \leftarrow \text{PARSETOOL}(r)$, $o \leftarrow t(\cdot)$ ▷ deterministic tool executes
- 7: **append** (t, o) to H
- 8: **return** assembled tool set

The experiments separate three sources of difficulty. The intent benchmark removes waveform processing and tests whether a controller can infer the appropriate tool set from operator language. Validity probes test sensitivity to labels, dataset, and scenario authoring. The closed-loop probe restores sequential tool execution and asks whether the controller should choose the final corrective action. Accurate tool selection does not imply reliable numerical control.

A. Intent to Tool Plan

Base intents are near saturation for both controllers and routers; novel and compound intents provide the main separation. Controller novel F1 ranges from 0.856 (Mistral-7B) to 0.921 (GPT-5.5). Among local models, Qwen3-30B-A3B reaches 0.902, Qwen3-8B reaches 0.895 [0.836, 0.947], and Gemma-4-12B reaches 0.876. All exceed the best single router (0.567) and the compound-trained router (0.649).

This difference reflects composition rather than lexical similarity. For an intent requesting a full health report without model adaptation, the nearest-neighbor router selects

only CHECKACCURACY. The controller selects CHECKACCURACY, DETECTDRIFT, PROBECONFUSION, and REJECTOPENSET, while omitting ADAPT. Relative to the pre-specified best router (char- n -gram NN), the F1 improvement is 0.289–0.334; all bootstrap 95% intervals exclude zero, with the narrowest interval [0.202, 0.381]. The per-intent best-of-six router reaches 0.613, serving as an oracle upper bound over the router family. A router trained with base intents plus half of the novel set in 2-fold cross-validation still trails the controller, suggesting that the improvement is not explained solely by missing compound examples.

The same pattern appears for consequential tools (Fig. 3b). Omitting ADAPT leaves an unadapted recognizer, and omitting REJECTOPENSET allows unknown signals to enter downstream decisions. The controller reaches recall 0.89 for ADAPT and 1.00 for REJECTOPENSET; the best router reaches 0.53 and 0.39 on the same tools. Spurious-call rates for the controller remain 0.05 and 0.18, respectively.

B. Validity

We apply three checks for curation and baseline fairness. First, the zero-shot tool-description router scores 0.477, and the semantic-embedding router scores 0.501 on Novel and 0.469 on Ext. Both are weaker than the trained routers. The controller exceeds all six router variants, indicating that the improvement is not tied to a single weak baseline.

Second, GPT-5.5 and Gemma-12B independently relabel the novel intents without seeing the author labels. Neither model is the evaluated 8B controller. Across the authors and the two models, Fleiss' κ is 0.894, and pairwise Cohen's κ ranges from 0.883 to 0.908. The controller remains ahead under every label

TABLE III

TOOL-SET F1 ($\uparrow$). NOVEL IS AUTHOR-WRITTEN HELD-OUT INTENTS; EXT IS GPT-5.5-AUTHORED INTENTS SCORED AGAINST ITS OWN LABELS.

Method	Base	Novel	Ext
fixed	0.219	0.263	0.250
router-nn (TF-IDF NN)	1.000	0.514	0.475
router-ml (TF-IDF multi-label)	0.974	0.507	0.475
router-char (char NN)	1.000	0.567	0.579
router-ml-char (char multi-label)	0.952	0.567	0.579
router-desc (zero-shot, tool-desc)	0.846	0.477	0.673
router-emb (8B embed, NN/ML)	–	0.501	0.469
best router (per-intent max)	–	0.613	0.673
Controller (GPT-5.5)	–	0.921	–
Controller (Qwen3-30B-A3B)	0.930	0.902 [0.852, 0.945]	–
Controller (Qwen3-8B)	0.991	0.895 [0.833, 0.947]	0.966
Controller (Gemma-4-12B)	0.965	0.876 [0.813, 0.932]	–
Controller (Mistral-7B)	0.982	0.856 [0.801, 0.907]	–

set, scoring 0.875–0.902 compared with 0.585–0.601 for the best router.

Third, we evaluate 16 intents *authored by* GPT-5.5 and scored against its own labels. The controller reaches 0.966, while the best router reaches 0.673. We treat this as a self-consistency probe rather than a generalization test, because GPT-5.5 acts as both author and grader. The main claim relies on author labels and the Qwen3-8B controller, neither of which depends on GPT-5.5.

These checks make the comparison conservative. The routers receive the same intent strings and tool inventory, and the per-intent best router is an oracle-style upper bound over six router designs. The relevant question is therefore whether a controller can recover unseen conjunctions such as health reporting, novelty screening, and adaptation. The LLM improvement is concentrated in this compositional regime.

C. Closed-Loop Boundary

We run a closed-loop probe with 8 hidden scenarios (4 base + 4 compound) and 5 seeds, giving $L = 40$ trials per model. The measured rate is

$$R_{\text{act}} = \frac{1}{L} \sum_{\ell=1}^L \mathbb{1}_{\{\hat{a}_{\ell} = a_{\ell}^*\}}. \quad (4)$$

Here $\hat{a}_{\ell}$ is the predicted action and a_{ℓ}^* the oracle action. When the corrective *action* must be inferred from numerical tool outputs, Qwen3-30B-A3B reaches $R_{\text{act}} = 0.375$ and Qwen3-8B reaches 0.350. Mistral-7B matches the fixed-pipeline baseline at 0.250, while Gemma-4-12B falls below it at 0.200. We also inject a simulated first-call failure and measure self-correction: Qwen3-30B-A3B reaches 0.375, Gemma-4-12B reaches 0.125, and the other two models remain at zero. All Clopper–Pearson 95% intervals overlap the baseline (Table IV), so no model significantly outperforms deterministic rules at $n=40$.

Only the two Qwen models exceed the point-estimate baseline, a scaling pattern not observed in intent planning. The dominant failure mode is novelty: no model reliably produces the final *flag* action, despite near-perfect REJECTOPENSET recall in the intent benchmark. The closed-loop task requires deciding *whether* novelty is present, not merely *which tool*

TABLE IV

CLOSED-LOOP CORRECT-ACTION RATE (8 SCENARIOS $\times$ 5 SEEDS = 40 TRIALS PER MODEL).

Method	Correct-action	Self-correction
fixed pipeline	0.250	–
learned router	0.250	–
ReAct (Mistral-7B)	0.250	0.00
ReAct (Gemma-4-12B)	0.200	0.125
ReAct (Qwen3-8B)	0.350 [0.21, 0.52]	0.00
ReAct (Qwen3-30B-A3B)	0.375 [0.23, 0.54]	0.375

should be called. We therefore implement final actions as a deterministic rule over tool returns,

$$\rho(o_{1:m}) = \begin{cases} \text{FLAG}, & s_{\text{ood}} > \tau_{\text{ood}}, \\ \text{ADAPT}, & \Delta_{\text{acc}} > \tau_{\text{drift}}, \\ \text{HOLD}, & \text{otherwise}, \end{cases} \quad (5)$$

where s_{ood} and Δ_{acc} come from REJECTOPENSET and DETECTDRIFT; thresholds are fixed outside the LLM and evaluated top-down. The evidence supports an architectural boundary: the LLM selects investigations, while verified tools and rules execute thresholded decisions.

This boundary also aids deployment. A planner can be audited by checking proposed calls against the fixed API, while the rule layer can be audited by replaying tool returns against thresholds. New false-alarm targets or validated diagnostic tools can then be handled by retuning rules or exposing a new text contract, without retraining the recognizer.

D. Case Study

In a representative drift case, CHECKACCURACY reports an accuracy of 0.61 against a 0.78 reference, DETECTDRIFT identifies covariate drift with a 0.17 accuracy drop, and ADAPT raises accuracy from 0.61 to 0.69. The sequence verifies the degradation, attributes it to drift, and invokes the corrective tool. A fixed pipeline adapts unconditionally, whereas a learned router selects a single tool. Under novelty rather than drift, the same controller calls REJECTOPENSET before adaptation, using the State Card to alter the plan without a separate hand-coded branch.

V. DISCUSSION AND LIMITATIONS

Conventional AMR systems classify incoming frames but rarely specify how the receiver should respond when operating conditions change. The proposed controller adds an adaptive control layer without modifying the recognizer, so the contribution is *orthogonal* to closed-set accuracy. The results should not be read as RF-waveform understanding: the controller is denied waveform access. Errors are also localized, since an incorrect tool set is a planning error, whereas threshold crossings, OOD scores, and adaptation effects remain attributable to deterministic tools.

The compositional improvement appears across 7B–30B models and three model families. Qwen3-30B-A3B (3B active) scores highest on novel compounds (0.902) but lowest on base intents (0.930), suggesting a trade-off between one-tool recall

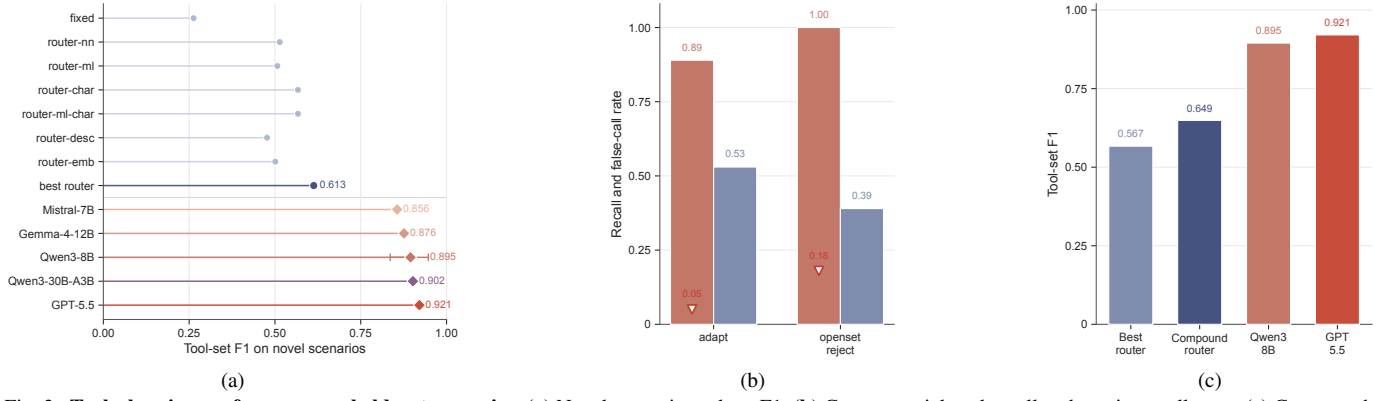


Fig. 3. **Tool-planning performance on held-out scenarios.** (a) Novel-scenario tool-set F1. (b) Consequential-tool recall and spurious-call rates. (c) Compound-trained router versus representative LLM controllers.

and larger-capacity composition. Dense 7–8B models show the opposite pattern.

The State Card in (1) is a compact interface rather than a complete world model. It may need to expand for multi-antenna channels, bursty interferers, or adversarial emitters. The benchmark is also focused: it contains 78 main scenarios and 16 cross-model scenarios, and it is author-curated despite blind relabeling (Fleiss' $\kappa = 0.894$). Future work should scale to practitioner-authored scenarios, live software-defined radios, and experience memory over past (state, plan, outcome) episodes.

Two deployment limitations remain. The planner assumes trustworthy card fields, and the tool API is intentionally small and closed. Larger systems will require versioned tool descriptions, return-value schemas, and prompt regression tests when tools are added or retired.

VI. CONCLUSION

This work studies LLM-based control for online AMR under a strict information boundary: the LLM plans over tools, the CNN performs recognition, and deterministic modules execute numerical decisions. Under this division, all five controllers improve novel-scenario tool-set F1 by at least 0.289 over the best router, with scores from 0.856 to 0.921. The result persists under blind relabeling, two datasets, cross-model scenario authoring, and three model families. The closed-loop probe further clarifies the boundary: numerical actions should remain deterministic, while the LLM composes the adaptive workflow.

REFERENCES

- [1] T. J. O'Shea, J. Corgan, and T. C. Clancy, "Convolutional radio modulation recognition networks," in *Engineering Applications of Neural Networks (EANN)*, ser. CCIS, vol. 629. Springer, 2016, pp. 213–226.
- [2] J. Xu, C. Luo, G. Parr, and Y. Luo, "A spatiotemporal multi-channel learning framework for automatic modulation recognition," *IEEE Wireless Commun. Lett.*, vol. 9, no. 10, pp. 1629–1632, 2020.
- [3] Z. Zhang, H. Luo, C. Wang, C. Gan, and Y. Xiang, "Automatic modulation classification using CNN-LSTM based dual-stream structure," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 13 521–13 531, 2020.
- [4] Y. Lin, Y. Tu, Z. Dou, L. Chen, and S. Mao, "Contour stella image and deep learning for signal recognition in the physical layer," *IEEE Transactions on Cognitive Communications and Networking*, vol. 7, no. 1, pp. 34–46, 2021.
- [5] Y. Tu, Y. Lin, H. Zha, J. Zhang, Y. Wang, G. Gui, and S. Mao, "Large-scale real-world radio signal recognition with deep learning," *Chinese Journal of Aeronautics*, vol. 35, no. 9, pp. 35–48, 2022.
- [6] J. Zhang, T. Wang, Z. Feng, and S. Yang, "AMC-Net: An effective network for automatic modulation classification," in *Proc. IEEE ICASSP*, 2023.
- [7] N. Soltani, J. Zhang, B. Salehi, D. Roy, R. Nowak, and K. Chowdhury, "Learning from the best: Active learning for wireless communications," *IEEE Wireless Commun. Lett.*, 2022.
- [8] S. Chen, Y. Zu, Z. Feng, S. Yang, and M. Li, "RadioLLM: Introducing large language model into cognitive radio via hybrid prompt and token reprogrammings," *arXiv preprint arXiv:2501.17888*, 2025.
- [9] M. Rostami, A. Faysal, R. G. Roshan, H. Wang, N. Muralidhar, and Y.-D. Yao, "Large language models can perform automatic modulation classification via discretized self-supervised candidate retrieval," *arXiv preprint arXiv:2510.00316*, 2025.
- [10] H. Zou, B. Wang, Y. Tian, L. Bariah, C. Huang, S. Lasaulce, and M. Debbah, "Seeing radio: From zero RF priors to explainable modulation recognition with vision language models," *arXiv preprint arXiv:2601.13157*, 2026.
- [11] T. B. Brown *et al.*, "Language models are few-shot learners," in *Proc. NeurIPS*, 2020.
- [12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Int. Conf. Learning Representations (ICLR)*, 2023, arXiv:2210.03629.
- [13] G. Wang, Y. Xie, Y. Jiang, A. Mandelkar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Trans. Mach. Learn. Res. (TMLR)*, 2023, arXiv:2305.16291.
- [14] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhamri, L. P. Saldyt, and A. B. Murthy, "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proc. Int. Conf. Machine Learning (ICML)*, ser. PMLR 235, 2024, pp. 22 895–22 907.
- [15] J. Tong, J. Shao, Q. Wu, W. Guo, Z. Li, Z. Lin, and J. Zhang, "WirelessAgent: Large language model agents for intelligent wireless networks," *arXiv preprint arXiv:2409.07964*, 2024.
- [16] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting large language models for networking," in *Proc. ACM SIGCOMM*, 2024, arXiv:2402.02338.
- [17] J. Ke, Q. Hu, S. Yuan, Y. Xu, and J. Yang, "SignalLLM: A general-purpose LLM agent framework for automated signal processing," *arXiv preprint arXiv:2509.17197*, 2025.
- [18] H. Chen, X. Liu, J. Jiang, B. He, and X. Liu, "Outcome-aware tool selection for semantic routers: Latency-constrained learning without LLM inference," *arXiv preprint arXiv:2603.13426*, 2026.
- [19] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.
- [20] A. Q. Jiang *et al.*, "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023.
- [21] Gemma Team, "Gemma 3 technical report," *arXiv preprint arXiv:2503.19786*, 2025.
- [22] T. J. O'Shea, T. Roy, and T. C. Clancy, "Over-the-air deep learning based radio signal classification," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 168–179, 2018.